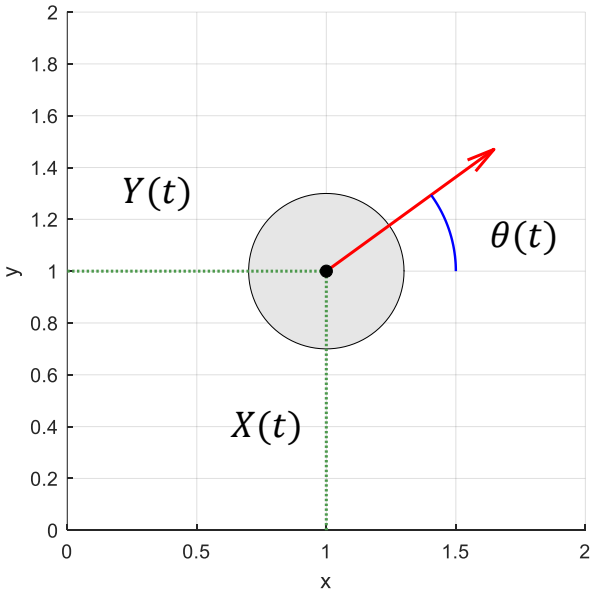
Cvičenie: Dynamická optimalizácia (optimálne riadenie mobilného robota)

Optimalizácia a optimálne riadenie

Problém:

- Mobilný robot s diferenciálnym podvozkom sa pohybuje v 2D priestore



- Uvažujme jeho kinematický model v spojitom tvare

$$\dot{X}(t) = v(t)\cos(\theta(t))$$

$$\dot{Y}(t) = v(t)\sin(\theta(t))$$

$$\dot{\theta}(t) = \omega(t)$$

  - $X(t), Y(t)$ sú jeho súradnice v priestore
  - $\theta(t)$ je uhol natočenia (orientácia)
  - $v(t)$ je dopredná rýchlosť pohybu
  - $\omega(t)$ je uhlová rýchlosť otáčania

- Stavový vektor a vektor vstupov bude:

$$x(t) = \begin{pmatrix} X(t) \\ Y(t) \\ \theta(t) \end{pmatrix} \quad u(t) = \begin{pmatrix} v(t) \\ \omega(t) \end{pmatrix}$$

- Jakobiány modelu môžeme odvodiť ako

$$\frac{\partial f(x(t),u(t))}{\partial x(t)} = \begin{pmatrix} 0 & 0 & -\sin(\theta(t))v(t) \\ 0 & 0 & \cos(\theta(t))v(t) \\ 0 & 0 & 0 \end{pmatrix} \quad \frac{\partial f(x(t),u(t))}{\partial u(t)} = \begin{pmatrix} \cos(\theta(t)) & 0 \\ \sin(\theta(t)) & 0 \\ 0 & 1 \end{pmatrix}$$

- Cieľom optimálneho riadenia je minimalizovať účelovú funkciu

$$J(x(t),u(t)) = \int_0^T L(x(t),u(t))dt = \frac{1}{2}\int_0^T x(t)^T Q x(t) + u(t)^T R u(t)\ \mathrm{d}t$$

- Váhovacie matice zvolíme ako diagonálne

$$Q = \begin{pmatrix} q & 0 & 0 \\ 0 & q & 0 \\ 0 & 0 & 0 \end{pmatrix},\ R = \begin{pmatrix} r_v & 0 \\ 0 & r_\omega \end{pmatrix}$$

- Uvažujem Pontryaginov princíp minima

$$\dot{\lambda}(t) = -\frac{\partial L(x(t),u(t))}{\partial x(t)} - \left(\frac{\partial f(x(t),u(t))}{\partial x(t)}\right)^T \lambda(t)$$

$$\frac{\partial L(x(t),u(t))}{\partial u(t)} = -\left(\frac{\partial f(x(t),u(t))}{\partial u(t)}\right)^T \lambda(t)$$

- Pre $L(x(t),u(t)) = \frac{1}{2}\left(x(t)^T Q x(t) + u(t)^T R u(t)\right)$ získame

$$\dot{\lambda}(t) = -Qx(t) - \left(\frac{\partial f(x(t),u(t))}{\partial x(t)}\right)^T \lambda(t)$$

$$Ru(t) = -\left(\frac{\partial f(x(t),u(t))}{\partial u(t)}\right)^T \lambda(t)$$

$$u(t) = -R^{-1}\left(\frac{\partial f(x(t),u(t))}{\partial u(t)}\right)^T \lambda(t)$$

- Dosadím Jakobiány a získavam

$$\begin{pmatrix} \dot{\lambda}_X(t) \\ \dot{\lambda}_Y(t) \\ \dot{\lambda}_\theta(t) \end{pmatrix} = -\begin{pmatrix} qX(t) \\ qY(t) \\ -\lambda_X(t)\sin(\theta(t))v(t) + \lambda_Y(t)\cos(\theta(t))v(t) \end{pmatrix}$$

$$\begin{pmatrix} v(t) \\ \omega(t) \end{pmatrix} = -\begin{pmatrix} \dfrac{\lambda_X(t)\cos(\theta(t)) + \lambda_Y(t)\sin(\theta(t))}{r_v} \\ \dfrac{\lambda_\theta(t)}{r_\omega} \end{pmatrix}$$

- Počiatočný stav $x(0)$ považujme za známy
- Koncový stav požadujme $x(T) = \begin{pmatrix} 0 \\ 0 \\ ? \end{pmatrix}$ pričom $\theta(T)$ je voľná (nezáleží nám na nej), preto $\lambda_\theta(T) = 0$
- Riešime príslušný Two Point Boundary Value Problem (TPBVP) pre koncový čas $T = 10$ s.

Robot bude prechádzať sekvenciou požadovaných bodov:

```
X_ref = [2, 1;
         1, -3;
        -2, -2;
        -4,  2;
         1,  3;
         2,  1];
```

```
opts = bvpset('RelTol', 1e-4, 'AbsTol', 1e-5, 'NMax', 5000);
```

Solver inicializujeme a používame nasledovne:

```
  % Set relative and absolute tolerances for the solver
    opts = bvpset('RelTol', 1e-4, 'AbsTol', 1e-5, 'NMax', 5000);

    % Initialize mesh points for solver and initial guess for solution
    solinit = bvpinit(linspace(0, T, N_steps), @(t) mobile_robot_initial_guess(t,
x0_shifted, y0_shifted, theta0, T));

    % Define the ODE system as a function handle with current parameters
    ode = @(t, x_lambda) mobile_robot_dx_lambda(t, x_lambda, q, rv, romega);

    % Solve the boundary value problem (BVP)
    sol = bvp4c(ode, @(x_lambda_0,x_lambda_T)
mobile_robot_bcfun(x_lambda_0,x_lambda_T, x0_shifted, y0_shifted, theta0),
solinit, opts);


    % Extract solution time points and corresponding solution variables
    t = sol.x;        % Time vector
    x_lambda = sol.y;        % Solution matrix: rows correspond to variables

    % Extract state variables from solution
    x = x_lambda(1, :);      % x position over time
    y = x_lambda(2, :);      % y position over time
    theta = x_lambda(3, :);  % orientation over time

    % Extract costate variables (Lagrange multipliers from PMP)
    lambda_x = x_lambda(4, :);
    lambda_y = x_lambda(5, :);
    lambda_theta = x_lambda(6, :);
```

`solinit = bvpinit(x,yinit)` uses the initial mesh `x` and initial solution guess `yinit` to form an initial guess of the solution for a boundary value problem. You then can use the initial guess `solinit` as one of the inputs to `bvp4c` or `bvp5c` to solve the boundary value problem.

`sol = bvp4c(odefun,bcfun,solinit)` integrates a system of differential equations of the form $y' = f(x,y)$ specified by `odefun`, subject to the boundary conditions described by `bcfun` and the initial solution guess `solinit`. Use the `bvpinit` function to create the initial guess `solinit`, which also defines the points at which the boundary conditions in `bcfun` are enforced.

To code `odefun`, use the functional signature `dydx = odefun(x,y)` for a scalar `x` and column vector `y`. The return value `dydx` is a column vector of data type `single` or `double` that corresponds to $f(x,y)$. `odefun` must accept both input arguments `x` and `y`, even if one of the arguments is not used in the function.

To code `bcfun`, use the functional signature `res = bcfun(ya,yb)` for column vectors `ya` and `yb`. The return value `res` is a column vector of data type `single` or `double` that corresponds to the residual value of the boundary conditions at the boundary points.

For example, if $y(a) = 1$ and $y(b) = 0$, then the boundary condition function is

```
function res = bcfun(ya,yb)
res = [ya(1)-1
       yb(1)];
end
```

Úloha:

Použite skript *mobile_robot_Pontryagin_scr*

Analyzujte vplyv váhovacích parametrov q, r_v, r_ω a koncového času T

Analyzujte vplyv voľby počiatočného odhadu trajektórie $x(t), \lambda(t)$ vo funkcii

```
function X_lambda = mobile_robot_initial_guess(t,x0,y0,theta0,T)
```

Implementujte funkciu pre výpočet optimálneho akčného zásahu (priečinok *mobile_robot*):

```
function [v, omega] = mobile_robot_get_optimal_u(theta, rv, romega, lambda_x,
lambda_y, lambda_theta)
    % Linear velocity control input
    v =%TODO
    % Angular velocity control input
    omega =% TODO
end
```

Implementujte funkciu pravej strany diferenciálnej rovnice pre riešenie TPBVP (priečinok *mobile_robot*):

```
function dx_lambda = mobile_robot_dx_lambda(t, x_lambda, q, rv, romega)

    % Unpack state variables
    x_pos = x_lambda(1);
    y_pos = x_lambda(2);
    theta = x_lambda(3);

    % Unpack costate variables
    lambda_x = x_lambda(4);
    lambda_y = x_lambda(5);
    lambda_theta = x_lambda(6);

    % Compute optimal controls based on current state and costates
    [v, omega] = mobile_robot_get_optimal_u(theta, rv, romega, lambda_x, lambda_y,
lambda_theta);

    % System dynamics (robot kinematics)
    dx = %TODO
    dy =  %TODO
    dtheta = %TODO

    % Costate dynamics (derived from Hamiltonian system)
    dlambda_x = %TODO
    dlambda_y = %TODO
    dlambda_theta = %TODO

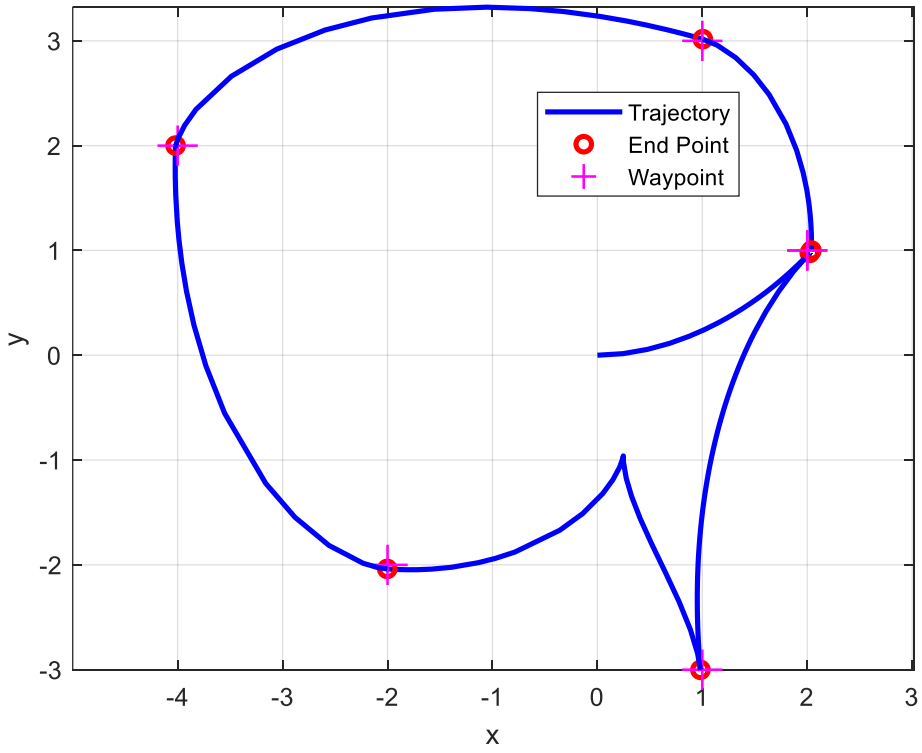
    % Pack derivatives into output vector
    dx_lambda = [dx; dy; dtheta; dlambda_x; dlambda_y; dlambda_theta];
end
```

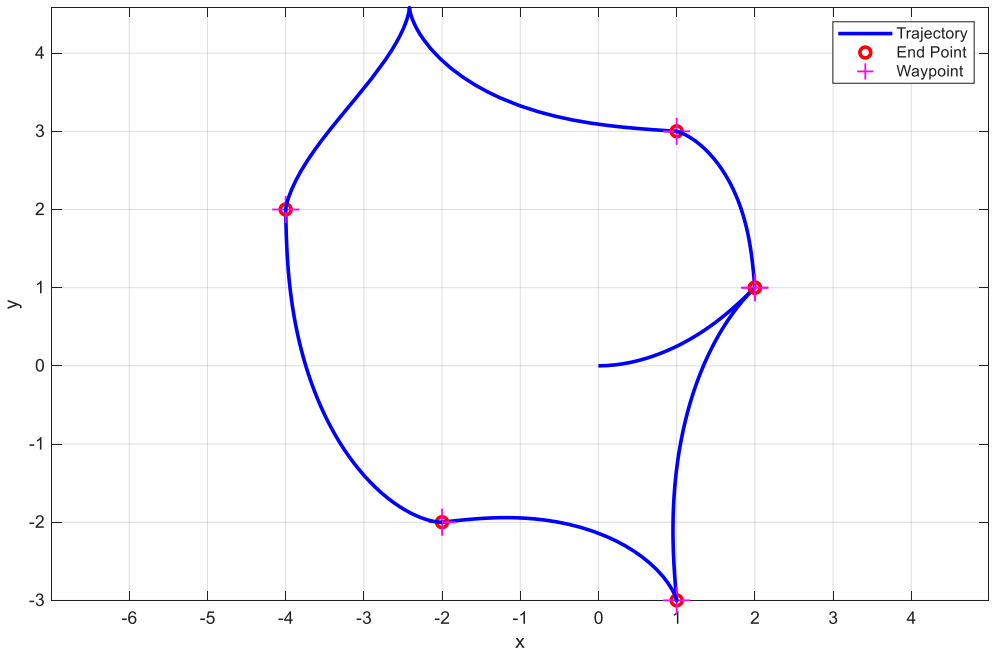
Implementujte funkciu pre výpočet chyby počiatočných a koncových pomienok TPBVP problému (priečinok *mobile_robot*):

```
function res = mobile_robot_bcfun(x_lambda_0,x_lambda_T, x0, y0, theta0)


    res = [%TODO;       % initial x position equals x0
           %TODO;       % initial y position equals y0
           %TODO;   % initial orientation equals theta0
           %TODO;            % terminal  x position equals 0
           %TODO;            % terminal  y position equals 0
           %TODO];           % terminal lambda_theta = 0
end
```

Riešenie:


Trajectory
End Point
Waypoint
x
y


Trajectory
End Point
Waypoint
x
y